UPES SCHOOL OF COMPUTER SCIENCE
DEPARTMENT OF INFORMATICS

Autonomous Research Report Agent (ARA)

Project Overview, Algorithmic Foundations & Viva Defense Manual

Comprehensive Preparation Guide for Mid-Term Evaluation & Beyond

---

Student Name	Enrollment No.	SAP ID	Specialization & Batch
Devansh Saini	R2142230351	500119031	B.Tech CSE (AIML), Batch 8
Yash Thakur	R2142230632	500119592	B.Tech CSE (AIML), Batch 8
Kanishq Vikram Singh	R2142230345	500121743	B.Tech CSE (AIML), Batch 7
Sumit Kumar	R2142230187	500119644	B.Tech CSE (AIML), Batch 7

Project Focus: Multi-Agent Automated Scientific Inquiry & Information Retrieval
Repository Branch: main (176 Passing Automated Tests with CI/CD)
Deliverables Included: Mid-Term Report (.pdf), Slide Deck (.pptx), Interactive CLI, Live Demo Harness

1. Executive Project Overview

The **Autonomous Research Report Agent (ARA)** is an evidence-grounded, multi-agent AI system designed to automate comprehensive academic literature reviews, technical due diligence, and scientific inquiry. Conventional language models suffer from severe operational failure modes when tasked with complex research: citation hallucination, confirmation bias (retrieving only confirmatory snippets), middle-context attention degradation (*'Lost in the Middle'*), and API rate-limiting bottlenecks.

ARA resolves these limitations through a disciplined four-phase pipeline where specialized autonomous agents collaborate under formal machine-readable JSON schema contracts:

Phase	Agent Role	Key Responsibilities	Emitted Contract
Phase 1	Research Planner	Domain detection, sub-question formulation, hypothesis generation, and falsification criteria.	data/research_plan.json
Phase 2	Discovery & IR Engine	Tri-provider retrieval (Tavily, OpenAlex, Semantic Scholar), multi-key deduplication, BM25 indexing, dense vector search, and sliding-window PDF passage chunking.	data/research_evidence.json
Phase 3	Evidence Validator & Critic	Sentence-level NLI audits, character-level quote provenance against raw PDF text, contradiction mapping, and iterative sufficiency re-search triggers.	data/validated_evidence.json *(Planned for Part 2)*
Phase 4	Calibrated Synthesizer	Epistemic confidence calibration (0.0 to 1.0), deterministic numerical citation links [1], [2], and multi-format report publishing (HTML, Markdown, PDF).	data/research_report.html *(Planned for Part 2)*

Key Defense Takeaway: For Mid-Term evaluation, **Phases 1 and 2 are fully engineered, tested, and demonstrated** with 176 unit tests. Phases 3 and 4 represent our planned development milestone for Major Project Part 2 (End-Term).

2. Scope Delineation: Mid-Term vs. End-Term

Examiners frequently ask: *'Where does your completed work end and your future work begin?'* The table below defines the clear architectural boundary:

Component / Dimension	Completed in Mid-Term (Part 1)	Planned for End-Term (Part 2)
Phase 1: Planning	• Domain classification (Biomedical, Technical, Legal) • Sub-question decomposition (Mechanism, Debate) • Working & competing hypothesis generation • Automated schema validation in planner/	• User-in-the-loop interactive plan editing via CLI • Dynamic ontology expansion for niche domains
Phase 2: Discovery & IR	• Tri-provider search (Tavily, OpenAlex, Semantic Scholar) • Hierarchical deduplication (DOI, arXiv ID, URL, Title) • 6-Step IR Engine (Porter stemmer + BM25 inverted index) • Two-Stage Cascade (500 -> 35 -> Top 10) • 350-word sliding-window PDF passage chunking	• Local PDF vector caching on disk (FAISS/Chroma integration) • Cross-lingual academic discovery support

Phase 3: Validation	• Character-level quote verification baseline in verifier/claim_verifier.py • Initial plan critique checks in critique/plan_critic.py	• Sentence-level Natural Language Inference (NLI) • Cross-study contradiction detection matrix • Sufficiency scorecard & automated iterative re-search loops
Phase 4: Synthesis & Output	• Interactive CLI with live spinners & Rich tables • Standalone pipeline demo (demo_semantic_pipeline.py) • Initial HTML report template with citation anchors	• Calibrated Epistemic Confidence scoring (0.0 to 1.0) • Headless PDF publication via WeasyPrint • Direct BibTeX and Zotero export integration
Engineering & QA	• 176 automated pytest suites passing in ~10.5s • Continuous Integration via .github/workflows/tests.yml	• End-to-end benchmark against standard RAG baselines on public academic benchmarks (HotpotQA, BioASQ)

3. Algorithmic Deep-Dive (Algorithms Implemented in Report)

This section details every algorithmic routine documented in Section 4 of the Mid-Term Evaluation Report. Expect exact questions on these mathematical formulations.

3.1 Robertson–Spärck Jones BM25 Probabilistic Relevance Ranking

Source Module: search_agent/semantic_indexer.py | **Benchmark Execution:** 0.021s on local CPU for 500 candidates.
BM25 is a non-linear term-matching ranking function that models the probability of relevance of candidate document *D* against user sub-question query *Q*. Unlike naive TF-IDF, BM25 prevents term frequency saturation and penalizes unnaturally long documents.

```
BM25 Formula:
Score(D, Q) = SUM_{i=1..n} [ IDF(q_i) * ( f(q_i, D) * (k_1 + 1) ) / ( f(q_i, D) + k_1 * ( 1
 - b + b * (|D| / avgdl) ) ) ]

Inverse Document Frequency (IDF):
IDF(q_i) = ln( ( N - n(q_i) + 0.5 ) / ( n(q_i) + 0.5 ) + 1 )
```

Core Parameters & System Values:

- **k■ = 1.5:** Term frequency saturation parameter. It controls how quickly the score asymptotes as term frequency *f(q_i, D)* increases. A higher *k■* rewards repeated keywords; 1.5 prevents keyword stuffing from dominating.
- **b = 0.75:** Document length normalization factor. If *b = 1*, full document length scaling is enforced; if *b = 0*, length is ignored. At 0.75, documents longer than *avgdl* are penalized unless they contain multiple distinct query terms.
- **N:** Total number of candidate documents in the current retrieval pool (typically ~500).
- **n(q_i):** Count of candidates containing stemmed token *q_i*.

3.2 Pure-Python Porter Stemming Algorithm (Porter, 1980)

Source Module: search_agent/semantic_indexer.py | **Standard:** Porter (1980) 5-Stage Suffix Stripping.
To enable BM25 to match morphological variants across academic literature (e.g., *'retrieval'*, *'retrieved'*, *'retrieving'* → **'retriev'**), a pure-Python Porter stemmer was engineered from first principles. It sequentially executes:

- **Step 1a–1c:** Plural and past-participle stripping (*-sses → -ss, -ies → -i, -ing/-ed removal*).
- **Step 2:** Double suffix reduction (*-ational → -ate, -ization → -ize, -fulness → -ful*).
- **Step 3:** Complex suffix conversion (*-iciti → -ic, -ical → -ic, -ative → [null]*).
- **Step 4:** Derivational suffix removal based on syllable measure *m > 1* (*-ance, -ence, -able, -ible, -ment*).
- **Step 5a–5b:** Terminal *-e* and double-consonant cleanup (*-ll cleanup if m > 1*).

Key Defense Takeaway: Crucial Viva Point: Explain that writing this in pure Python guarantees **100% portability** across any operating system without requiring compiled C extensions, NLTK wheels, or spaCy runtime downloads.

3.3 Two-Stage Cascade Candidate Filtering (Coarse-to-Fine Search)

Problem Solved: Generating dense vector embeddings for 500+ candidates hits LLM embedding rate limits (HTTP 429) and induces 4-6 second latency spikes per query.
Mechanism: ARA implements a two-stage cascade architecture:

```
Stage 1 (Coarse Filter - Local CPU, 21ms):
  500 Gross Candidates ■■[BM25 Scoring + Porter Stemming]■■■ Top 35 Candidates

Stage 2 (Dense Re-Ranking - Gemini text-embedding-004 API):
  Top 35 Candidates ■■[Compute 768-dim Vector Embeddings]■■■ Cosine Similarity

Hybrid Fusion Formula:
  FinalScore(D) = alpha * Score_BM25_norm(D) + (1 - alpha) * CosineSim(v_Q, v_D)
  where alpha = 0.35 (Lexical Weight), 1 - alpha = 0.65 (Semantic Dense Weight)
```

Empirical Result: Reduces dense vector API requests by **93.0%** (from 500 API calls down to 35), completely eliminating rate limits while retaining 100% recall of relevant academic papers.

3.4 Sliding-Window PDF Passage Chunking

Source Modules: search_agent/content_retriever.py and search_agent/semantic_indexer.py.
Theoretical Justification: *Liu et al. (2024)* demonstrated in their foundational paper **'Lost in the Middle: How Language Models Use Long Contexts'** that LLMs exhibit a U-shaped retrieval curve: information placed in the middle 40%–60% of an input prompt suffers high retrieval error rates.
To prevent context dilution from 15-page arXiv PDFs, ARA applies sliding-window empirical chunking:

- **Window Size (W = 350 words):** Approximately 450 tokens, matching the typical length of a scientific methodology paragraph or experimental results section.
- **Step Size (S = 300 words) & Overlap (O = 50 words):** A 50-word overlap guarantees that critical quantitative findings (e.g., benchmark tables, confidence intervals) spanning sentence boundaries are not truncated.
- **Passage Selection Algorithm:** Each extracted passage is scored against the sub-question prompt using local BM25 density. Only the **top 5 high-density empirical paragraphs** are returned for evidence synthesis.

3.5 Multi-Key Hierarchical Deduplication Algorithm

Source Module: search_agent/sources.py | **Benchmark:** 92.2% retention efficiency.
Because academic literature appears concurrently across open-access repositories (arXiv), indexers (Semantic Scholar, OpenAlex), and commercial publishers under conflicting URLs, ARA executes a hierarchical resolution cascade:

```
Resolution Priority:
  1. Canonical DOI: Match normalized DOI string (e.g., '10.1145/3586183.3606763')
  2. Canonical arXiv ID: Strip version suffixes ('arXiv:2305.14283v2' -> '2305.14283')
  3. Canonical URL: Normalize scheme (https), strip trailing slash & UTM tracking params
  4. Fuzzy Title Matching: Levenshtein / Token-sort similarity >= 0.88 with author check
```

During live benchmark execution on 539 harvested candidates, the deduplication engine successfully merged 42 duplicate records, retaining 497 unique academic sources without data loss.

3.6 Logarithmic Citation Authority Dampening

Formula: AuthorityScore(D) = log10(1 + citations_D) / log10(1 + citations_max)
In naive ranking systems, seminal papers from 2017 with 40,000+ citations receive overwhelming authority scores, starving recent 2025/2026 preprints with 3-10 citations of visibility. Logarithmic scaling compresses the authority dynamic range, allowing high-relevance contemporary preprints to compete fairly against legacy foundations.

4. Team Viva Strategy & Division of Responsibility

To deliver a cohesive, confident presentation, each team member should own a specific pillar of the pipeline during the viva defense:

Team Member	Assigned Viva Pillar	Key Concepts & Modules to Own
Devansh Saini	Phase 1: Planning & Theoretical Problem Statement	• Problem formulation & RAG vulnerabilities (Hallucination, Lost in the Middle) • Domain detection heuristic & question decomposition • Working & competing hypotheses + falsification criteria • Codebase: planner/planner.py, planner/validation/

Yash Thakur	Phase 2: 6-Step IR Engine & BM25 Mathematics	• Robertson–Spärck Jones BM25 mathematical formulation (k1, b, IDF) • Pure-Python Porter stemmer (5 stages, zero external dependencies) • Inverted index construction & local CPU term scoring • Codebase: search_agent/semantic_indexer.py
Kanishq Vikram Singh	Two-Stage Filtering & PDF Passage Chunking	• Two-stage cascade rationale (500 -> 35 -> Top 10, >90% API reduction) • Sliding-window chunking (350w window, 50w overlap, pypdf parser) • 'Lost in the Middle' mitigation benchmark (Liu et al., 2024) • Codebase: search_agent/content_retriever.py, search_agent/source_ranker.py
Sumit Kumar	Discovery, Multi-Key Deduplication & Test Suite	• Tri-provider harvest (Tavily, OpenAlex, Semantic Scholar) • Multi-key deduplication cascade (DOI, arXiv ID, URL, Fuzzy Title) • 176 automated pytest tests & GitHub Actions CI/CD pipeline • Codebase: search_agent/sources.py, search_agent/source_providers.py, .github/

5. Top 10 Viva Defense Questions & Master Answers

These ten questions represent the most likely lines of inquiry from the examination committee. Rehearse these answers verbatim with your teammates:

Q1: Why did you build a custom BM25 and Porter stemmer instead of importing NLTK, Whoosh, or LangChain?

Answer: Heavy NLP libraries like NLTK or LangChain introduce bloated dependency trees, opaque prompt wrappers, and platform-specific compiled C extensions. Implementing BM25 and the 5-stage Porter stemmer in pure Python guarantees that ARA runs in 21ms on any local CPU with zero native binary requirements. It gives us full transparency, allows dynamic term weighting adjustments, and is validated by our own automated unit tests.

Q2: Modern LLMs (like Gemini 1.5 or Claude 3.5) support 1M+ token context windows. Why bother with 350-word passage chunking?

Answer: While long context windows exist, empirical research by Liu et al. (2024, 'Lost in the Middle') proved that retrieval accuracy degrades severely when facts are positioned within the middle 40%–60% of an input prompt. Stuffing 15-page PDFs also causes context dilution, high financial cost, and severe latency. Sliding-window chunking isolates the top 5 high-density empirical paragraphs, giving higher factual precision at a fraction of the token cost.

Q3: What specific empirical advantage does your Two-Stage Cascade give over standard dense vector retrieval?

Answer: In naive vector RAG, embedding 500 candidate documents requires 500 individual API calls, inducing high latency and immediately triggering HTTP 429 rate limits on production endpoints. Our two-stage cascade executes Stage 1 BM25 coarse filtering on local CPU in 0.021s, narrowing 500 candidates down to 35. Vector embeddings are generated solely for those 35, reducing API consumption by 93.0%.

Q4: How does your system prevent confirmation bias during retrieval?

Answer: Conventional search agents query engines using leading user prompts, returning only confirmatory snippets. In Phase 1, ARA explicitly formulates both working hypotheses and competing hypotheses with defined falsification criteria. In Phase 2, the query formulation engine generates dedicated counter-evidence queries specifically searching for dissenting empirical results.

Q5: How do you handle cases where two papers report completely contradictory findings?

Answer: In Phase 1, competing hypotheses are established to accommodate contradictory viewpoints. In Phase 2, evidence cards are tagged with empirical methodology tiers (Randomized Trial vs. Observational vs. Synthetic). In Phase 3 (End-Term roadmap), our contradiction detection matrix cross-references opposing effect sizes and highlights epistemic uncertainty in the final report rather than artificially smoothing over disagreements.

Q6: Why did you choose OpenAlex and Semantic Scholar alongside Tavily instead of just using Google/Bing search?

Answer: Commercial web search APIs index blog posts, marketing landing pages, and secondary commentary, often lacking peer-reviewed authority. OpenAlex and Semantic Scholar provide direct access to structured scholarly metadata: DOIs, publication years, citation graphs, open-access PDF URLs, and formal abstracts. Interleaving web discovery with academic graphs ensures both timeliness and peer-reviewed rigor.

Q7: How do you guarantee that citations in your final report are not hallucinated?

Answer: In conventional RAG, citation assignment is left to the LLM's non-deterministic completion, frequently resulting in fabricated links. In ARA, every cited passage is extracted as a structured evidence card containing an audited character-level substring from raw source text. Numbered citations [1], [2] are assigned deterministically to canonical DOI/arXiv links.

Q8: What is the purpose of the Logarithmic Citation Authority Dampening function?

Answer: If raw citation count is used in ranking, foundational papers with 50,000 citations (e.g., Vaswani et al., 2017) completely dominate the retrieval pool, starving relevant, cutting-edge 2025/2026 preprints of visibility. Logarithmic scaling (log10(1 + citations)) compresses the dynamic range, allowing recent breakthroughs with 5 citations to compete fairly.

Q9: How do you verify that your software works without relying on live API calls during viva?

Answer: ARA includes a deterministic offline demonstration harness (python main.py --demo) and a standalone semantic pipeline visualizer (python demo_semantic_pipeline.py). Furthermore, our repository contains 176 automated pytest tests that pass in ~10.5 seconds, integrated with GitHub Actions CI/CD to provide verifiable proof of code health.

Q10: What is your primary milestone for Major Project Part 2 (End-Term)?

Answer: Our primary Part 2 milestone is the complete realization of Phase 3 (Sentence-Level Natural Language Inference for claim auditing, contradiction detection, and autonomous sufficiency re-search loops) and Phase 4 (Calibrated Epistemic Confidence scoring from 0.0 to 1.0 and headless publication-grade PDF compilation via WeasyPrint).